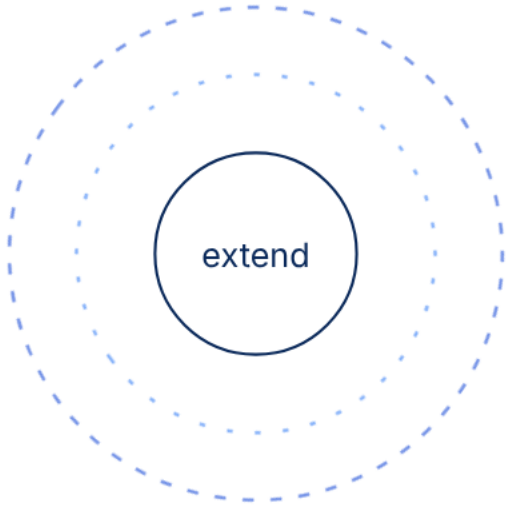


BUILDING PRODUCTS WITH CLAUDE · MODULE 3

Extending Claude Code: Skills, Hooks, Subagents and Plugins

What each mechanism is, how to create one, when to use which, how to test and package them, and how to keep the context small while doing so.

Manaranjan Pradhan
www.manaranjanp.com

OVERVIEW

Where the mechanisms sit around a session



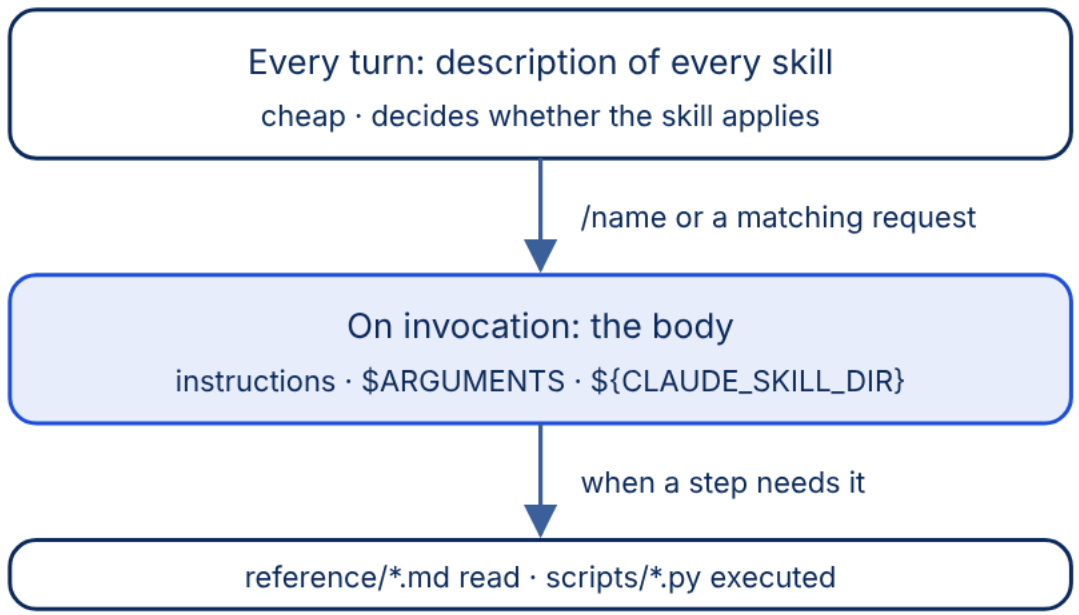
Who executes it: the model (CLAUDE.md, skill) · your shell (hook) · a second model (subagent) · an external process (MCP) · the installer (plugin).

DEFINITION

Skill: a procedure the model loads on demand

A folder with a `SKILL.md` file: YAML frontmatter plus Markdown instructions. Optional `scripts/` and `reference/` files beside it.

Lives in	`.claude/skills/<name>/` (project) · `~/.claude/skills/<name>/` (personal) · inside a plugin as `plugin:name`
Triggered by	You: `/name args`. The model: when the request matches the description.
Always in context	The description (with `when_to_use`), cut at 1,536 characters
Loaded on invocation	The body, for the rest of the session; scripts are executed, not loaded
Runs as	The model, following the steps; scripts run in your shell



DEFINITION

Hook: code that runs at an event, whatever the model decides

Defined in	`settings.json` (user, project, local, managed) · a plugin's `hooks/hooks.json` · a skill's or subagent's frontmatter
Events	`SessionStart` · `UserPromptSubmit` · `PreToolUse` · `PostToolUse` · `Stop` · `SubagentStop` · `PreCompact` · and others (30+)
Matcher	Tool name or list (`Edit|Write`); regex if other characters appear; empty = all. `if` narrows further with a permission rule: `Bash(git commit *)`
Handler types	`command` (deterministic) · `http` · `mcp_tool` · `prompt` / `agent` (a model decides; not for enforcement)
Input / output	Event JSON on stdin · exit 0 continue, exit 2 block with stderr as the reason · JSON out: `systemMessage` , `continue` , `hookSpecificOutput.permissionDecision`

Advice versus enforcement

A line in `CLAUDE.md` is advice the model weighs. A hook runs every time. If a rule must hold with zero exceptions, it is a hook or a permission deny rule, not a sentence.

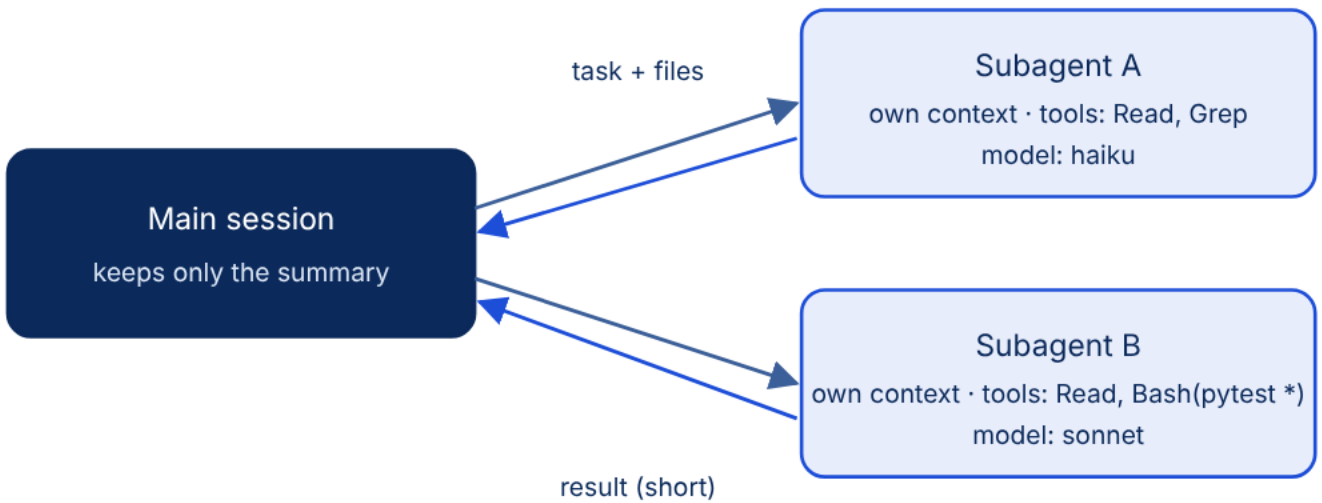
Runs as you

With your user's permissions, in your shell. Read every hook before installing one from elsewhere.

UI `/hooks` lists every hook and where it came from; editing is done in the files.

DEFINITION

Subagent: a second session with its own context, tools and model



File	`.claude/agents/<name>.md` · `~/.claude/agents/` · in a plugin
Frontmatter	`name` , `description` , `tools` , `disallowedTools` , `model` , `permissionMode` , `maxTurns` , `skills` , `memory` , `isolation: worktree` , `background`
Invoked by	The model when the description matches · you by name or `@name` · `claude --agent name`
Returns	Only its final message. Foreground blocks; background runs concurrently and reports later (Ctrl+B)
Built in	`Explore` , `Plan` , `general-purpose` , `claude-code-guide`

COST No shared context: the subagent sees only what the request and the files give it, and the main session sees only what it returns.

DEFINITION

Plugin: the distribution unit

```
ask-db-tools/
├── .claude-plugin/plugin.json   # name (required), version, author, description
├── skills/<name>/SKILL.md       # namespaced as ask-db-tools:name
├── agents/<name>.md
├── hooks/hooks.json             # {"hooks": {...}}
├── .mcp.json                    # servers the plugin starts
├── .lsp.json                    # language servers
└── bin/                         # added to PATH
```

Found through	A marketplace: a repository with `.claude-plugin/marketplace.json` listing `plugins[] {name, source}` ; sources pinned to a tag or SHA
Installed with	`/plugin install name@marketplace` at **user**, **project** (in `.claude/settings.json`, so the team gets it) or **local** scope
Tried with	`claude --plugin-dir ./ask-db-tools` : active for one session, nothing written to settings
Checked with	`claude plugin validate ./ask-db-tools` (`--strict` in CI)
Paths inside	`${CLAUDE_PLUGIN_ROOT}` (this version) · `${CLAUDE_PLUGIN_DATA}` (survives updates)

> NOTE A plugin adds nothing new: it packages skills, agents, hooks and servers so that installing them is one command and updating them is one version bump.

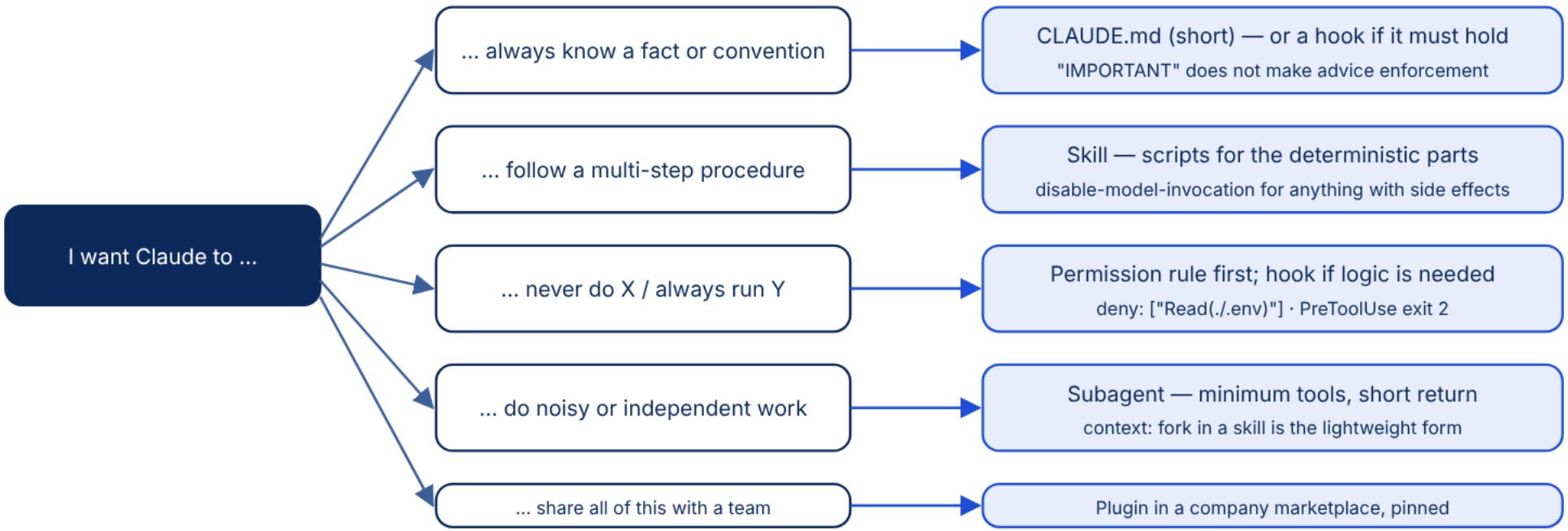
COMPARISON

The same six questions for each mechanism

	CLAUDE.md	Skill	Hook	Subagent	Plugin
What it holds	Standing guidelines	A procedure	A script at an event	A worker with its own context	Any of the others, packaged
Who runs it	The model, as advice	The model, step by step	Your shell, always	A second model	The installer
Trigger	Session start	`/name` or matching request	Lifecycle event	Delegation by description or name	`/plugin install`
Context cost	Whole file, every turn	Description every turn; body when used	None	None in the main session; its own window	Sum of its parts
Deterministic	No	No (scripts: yes)	Yes (`command`)	No	—
Use it for	Facts the model cannot derive	Repeatable team workflows	Rules with zero exceptions, progress display	Noisy or independent work, review	Sharing and versioning

DECISION

Which one to write



CREATING · SKILL

A minimal skill

```
# .claude/skills/sql-review/SKILL.md
---
name: sql-review
description: Checklist for reviewing SQL against an Ask DB
  database configuration. Use when asked to review, check or
  approve SQL, or by the sql-reviewer subagent.
argument-hint: <path-to-sql-or-yaml>
allowed-tools: Read Grep
---
Review the SQL in $ARGUMENTS against config/databases/*.yaml.

1. Only SELECT or WITH. Anything else: REWRITE.
2. Every table and column exists in the configuration.
3. Metric definitions are used as written, not re-derived.
4. Reply: APPROVE or REWRITE, then findings as
   `file:line — problem — fix`. Nothing else.
```

`name` → the `/sql-review` command; defaults to the folder name.

`description` → what and *when*; the trigger words users would say, key case first.

`$ARGUMENTS` → everything after the command; `$0`, `$1` or named `arguments:` for positions.

`allowed-tools` → tools that need no prompt during the invoking turn.

Other fields: `disable-model-invocation`, `user-invocable`, `model`, `effort`, `context: fork`, `hooks`, `paths`.

```
/reload-skills
/sql-review config/validation/cloudmetrics.yaml
/context all       # the skill and its token cost
```

CREATING · SKILL

A skill with scripts and reference files

```
.claude/skills/db-config/
├── SKILL.md              # the procedure, ~80 lines
├── reference/format.md   # the YAML format; read at step 3
└── scripts/
    ├── introspect.py     # URL → JSON (tables, keys, counts, samples)
    ├── render_config.py  # JSON → draft YAML with TODOs
    └── check_config.py   # YAML vs live database → PASS / FAIL

# SKILL.md, body
1. Run `python ${CLAUDE_SKILL_DIR}/scripts/introspect.py "$0"`.
2. Run render_config.py on the JSON. Open the draft.
3. Read reference/format.md. Resolve every TODO with the user:
   column meanings, grain, NULLs, which table answers what.
4. Run check_config.py until it prints PASS.
5. Hand over to metric-definitions.
```

Deterministic work goes in scripts

Introspection, rendering and checking are code. The model does not re-derive them each time; it runs them and reads the output. Scripts are tested with pytest like any other code.

Judgement stays in the body

What a column means, which metric definition is right, which tables to hide: questions for the user, asked by the model.

Reference files load on demand

`format.md` is read only at step 3; it costs nothing until then. Keep `SKILL.md` under 500 lines; most are far shorter.

CREATING · HOOK

A hook that blocks a class of commands

```
# .claude/settings.json
{ "hooks": {
    "PreToolUse": [ { "matcher": "Bash",
      "hooks": [ { "type": "command",
        "command": "python .claude/hooks/block_db_shell.py" } ] } ],
    "PostToolUse": [ { "matcher": "Edit|Write",
      "hooks": [ { "type": "command",
        "command": "python -m pytest -q -x tests | tail -3" } ] } ] } }

# .claude/hooks/block_db_shell.py
payload = json.load(sys.stdin)
cmd = payload.get("tool_input", {}).get("command", "")
if BLOCK.search(cmd) and not ALLOW.search(cmd):
    print("Blocked: use the app's query path or a skill script.",
          file=sys.stderr)
    sys.exit(2)        # 2 = block; stderr goes to the model
sys.exit(0)
```

PreToolUse on Bash: every shell command passes through the script before it runs. Exit 2 stops it and tells the model why.

PostToolUse on Edit|Write: the test suite runs after every file change; the tail of its output enters the context.

The rule here: the only path to a database is the application or a skill script. Direct `sqlite3`, `psql` or `.db` access from the shell is refused.

Test by hand before trusting it: `echo '{"tool_input":{"command":"sqlite3 x.db"}}' | python block_db_shell.py; echo $?` → 2.

> SIMPLER FIRST If the rule is a plain allow or deny on a tool pattern, a permission rule does it with no script.

CREATING · SUBAGENT

A read-only reviewer

```
# .claude/agents/sql-reviewer.md
---
name: sql-reviewer
description: Reviews SQL in validation cases, metric definitions
  and generated queries against the database configuration.
  Use after eval-dataset or db-config writes a file.
tools: Read, Grep, Glob, Bash(python .claude/skills/*/scripts/*)
model: sonnet
skills: sql-review
---
You review; you do not edit.
Apply the sql-review checklist to every SQL statement in the
file you are given. Run run_sql.py to execute each one.
Reply with VERDICT, then at most ten findings as
`case-id — problem — fix`. No summary of the file.
```

`tools` is an allowlist: no Edit, no Write, Bash only for the skill scripts. The reviewer cannot change what it reviews.

`skills: sql-review` injects the checklist at start-up; the subagent does not have to find it.

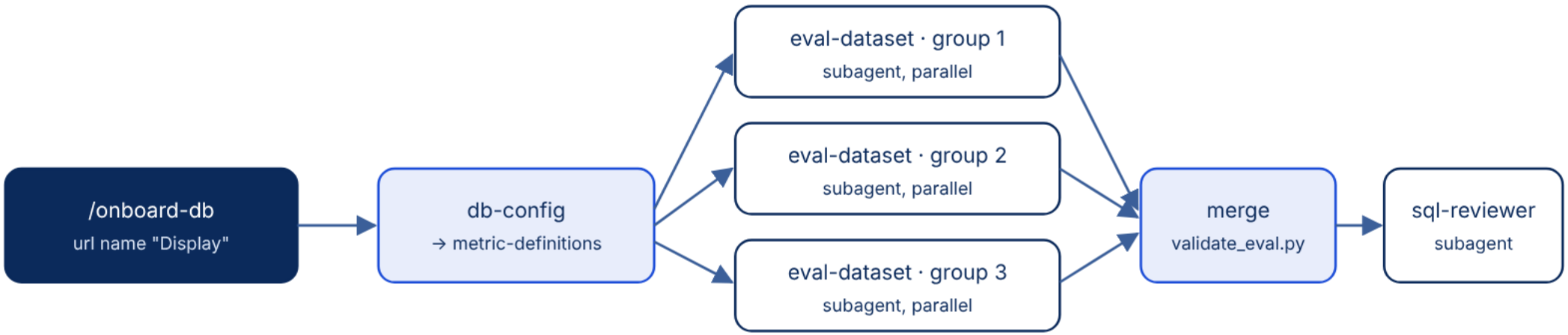
`model: sonnet` : judgement without the top-tier cost; `haiku` for fan-out jobs.

The last paragraph is the return contract. The main session acts on a verdict and a list, not on prose.

Invoke: "run the sql-reviewer on config/validation/cloudmetrics.yaml", or automatically when the description matches.

PARALLEL A skill can launch several subagents at once (one per table group in `onboard-db`) and merge what they return.

Composing them: one command onboards a database



Hooks run throughout: PreToolUse blocks direct database access; PostToolUse runs the tests after each write.

Outputs: config/databases/<name>.yaml and config/validation/<name>.yaml, both PASS, plus the reviewer's verdict. Upload through the admin UI.

CREATING · PLUGIN

Packaging and installing

```
# ask-db-tools/.claude-plugin/plugin.json
{ "name": "ask-db-tools", "version": "1.0.0",
  "description": "Onboarding, validation and review skills for Ask DB",
  "author": { "name": "Ask DB team" } }

# marketplace.json (in a company repository)
{ "name": "acme-tools", "owner": { "name": "Acme" },
  "plugins": [ { "name": "ask-db-tools",
      "source": { "source": "github", "repo": "acme/ask-db-tools",
                  "ref": "v1.0.0" } } ] }
```

```
claude plugin validate ./ask-db-tools       # structure
claude --plugin-dir ./ask-db-tools          # trial, one session
/plugin marketplace add acme/acme-tools
/plugin install ask-db-tools@acme-tools --scope project
/reload-plugins
/ask-db-tools:db-config sqlite:///data/x.db x
```

Project scope writes the plugin into `.claude/settings.json`; teammates get it at their next session.

Pin the source to a tag or SHA; upgrades are a reviewed change to the marketplace file.

`extraKnownMarketplaces` in settings registers the company marketplace for everyone.

PRACTICE · SKILLS

Descriptions decide whether a skill fires

Too broad

description: Helps with database work.
Fires on every SQL question, competes with other skills, and wastes the body's tokens on tasks it was not written for.

Specific

description: Create or refresh the database configuration YAML (connection, schema with column descriptions, business rules) for an Ask DB database from its SQLAlchemy URL. Use when asked to onboard a database, document a database for Ask DB, generate the config YAML, or refresh it after a schema change.

State **what** it produces and **when** to use it, with the words users say. Key use case first; the listing is cut at 1,536 characters.

Name the negative space when siblings overlap: "not for writing or reviewing SQL; see sql-review".

Side effects or long runs → `disable-model-invocation: true`; the description then leaves the context too.

Test with eval cases: three prompts that should trigger it, three that should not. Re-run after every description change.

PRACTICE · SKILLS

Writing the body

Short and imperative

Numbered steps, one action each. Key instructions at the top: compaction keeps the first 5,000 tokens of a skill. Under 500 lines; aim for under 100.

Scripts for facts, model for judgement

Parsing, introspection, validation, API calls: `scripts/`, run with `${CLAUDE_SKILL_DIR}`. Questions, naming, trade-offs: steps that say "ask the user".

Reference on demand

Formats, examples, long tables: `reference/*.md`, read at the step that needs them. The body says *when* to read them.

Explicit end state

"Finish only when check_config.py prints PASS." "Stop. Do not start the next slice." A skill without a stop condition keeps going.

Hand-offs by name

"Then run metric-definitions." Chains of small skills beat one large one: each is testable and each loads only when used.

Least tools

`allowed-tools: Bash(python ${CLAUDE_SKILL_DIR}/scripts/*) Read Write`, not `Bash(*)`. The frontmatter documents the blast radius.

PRACTICE · SUBAGENTS

Writing subagents

One job per agent. "Reviews SQL against the configuration", not "helps with databases".

Read-only by default. Add Edit, Write or Bash only when the job is to change something, and scope Bash to patterns.

Cheapest model that does the job. `haiku` for search and fan-out, `sonnet` for review, the top tier for design.

A return contract. Say the format and the maximum length of the reply. The main session pays for every token that comes back.

Give it what it needs. File paths, the configuration, the case ids: it has no memory of your conversation. `skills:` injects a checklist at start.

Bound it. `maxTurns` for runaway loops; `isolation: worktree` when it edits; `background: true` when you do not need to wait.

Use a subagent when

The work is noisy (large reads, test output) · needs independence (review of the main session's work) · parallelises (one per table group) · needs different tools or a different model.

Do not use one when

The task needs the conversation so far · a single call would do (a script, a skill step) · the result must be a large artefact (write it to a file and return the path).

PRACTICE · HOOKS

Writing hooks

Fast. A PostToolUse hook runs after every edit; two seconds there is two seconds on every edit. `-x`, `tail`, targeted test selection.

Narrow matchers. `Edit|Write`, not empty; `if: "Bash(git commit *)"` when only one command matters.

Permission rules before hooks. Plain allow or deny of a tool pattern needs no script.

`command` for enforcement. `prompt` and `agent` handlers ask a model; they are not deterministic.

Exit codes are the interface. 0 continue, 2 block with a one-line reason on stderr that the model can act on.

Test by hand with a JSON payload on stdin before relying on it; keep the script under `.claude/hooks/` and in version control.

Portable paths. `$CLAUDE_PROJECT_DIR`; LF line endings for `.sh` on Windows.

Display	Stop → progress line (`systemMessage`)
Enforce	PreToolUse → exit 2 on a forbidden command
Verify	PostToolUse on Edit|Write → run tests or a formatter
Prepare	SessionStart → export variables, print branch and plan status
Gate	PreToolUse + `if: Bash(git commit *)` → tests must pass before a commit

KEEP IT SMALL Every hook is a tax on every matching event. Three or four well-chosen hooks per project is typical.

TOOLING

Tools for writing, checking and measuring extensions

skill-creator (anthropics/skills)	Interview-driven creation and iteration of a skill: with-skill versus without-skill benchmarks, blind A/B of two versions, description tuning with should- and should-not-trigger prompts, HTML review viewer.
plugin-dev (official marketplace)	Scaffolding and reference skills for plugins, hooks and skills; the hooks skill is a good model for your own.
claude plugin validate <dir>	Checks the manifest, hooks.json and layout. `--strict` turns warnings into failures for CI.
claude plugin eval <dir>	Runs the plugin's eval cases in a sandbox; pass rate against a threshold; JSON and HTML reports.
/skill-doctor	Per-skill context cost and invocation count; flags skills never used.
/doctor · /doctor prompt-audit	Unused skills, plugins and servers with their cost; slow hooks; CLAUDE.md duplication. prompt-audit finds outdated or conflicting instructions.
/context all	What every memory file, skill, agent and server costs right now.
/hooks · /mcp · /plugin	What is registered, from where, and whether it is enabled.
agnix (community)	A linter for CLAUDE.md, AGENTS.md, SKILL.md, hooks and MCP configuration, with autofixes.
pytest on scripts/	Skill scripts are code; test them against a sample database like any other code.

TOOLING · EVALUATION

Evaluating a skill: does it trigger, and only then?

```
evals/
├── db-config-plain-request/
│   ├── prompt.md        # "Onboard sqlite:///data/cm.db as cloudmetrics"
│   └── graders/
│       └── skill-used.md
└── db-config-not-readme/
    ├── prompt.md        # "Write a README for this repo"
    └── graders/
        └── skill-not-used.md

# graders/skill-used.md
---
type: tool_used
tool: Skill
input_match: '"skill"\s*:\s*"(?:[\w-]+:)?db-config"'
---
# graders/skill-not-used.md: same, with min: 0 and max: 0
```

Grader types	`tool_used` , `tool_order` , `regex` , `file_exists` (free) · `llm` , `baseline` (a judge; costs tokens)
Run	`claude plugin eval . --runs 3 --threshold 0.8 --json`
Results	`evals/results/<ts>/aggregate-result.json` and `report.html` ; exit 0 pass, 1 below threshold
CI	A workflow job on pull requests that touch `.claude/skills/` ; `--trust-plugin` for unattended runs

Three should-trigger and three should-not-trigger cases per skill is enough to catch most description regressions.

The should-not cases are the important ones: a skill that fires on "write a README" costs every session.

Deterministic graders first; an `llm` grader only for output quality.

GUIDELINES

Do and do not

Do

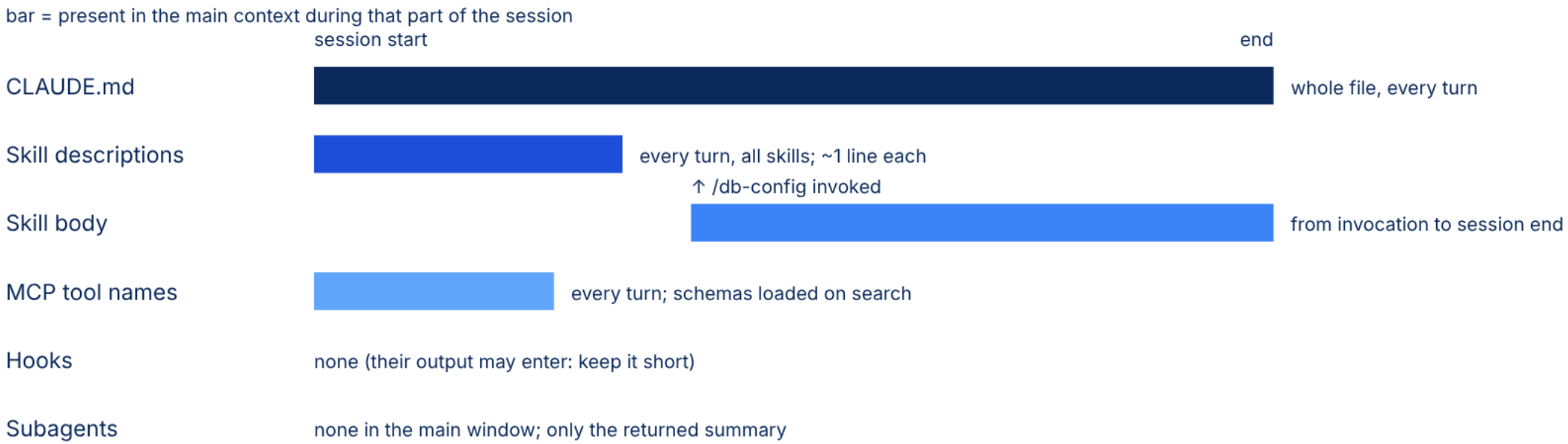
- Start with the smallest mechanism: a permission rule before a hook, a skill before a subagent, a file before a plugin.
- Write the description before the body; test it with should and should-not cases.
- Put deterministic work in scripts and test the scripts.
- Give every skill a stop condition and every subagent a return format.
- Measure context before and after (`/context all`); disable what is not used.
- Version in Git; pin plugin sources; review upgrades as pull requests.
- Read every hook and script before installing someone else's.

Do not

- Put rules that must always hold in `CLAUDE.md` ; "IMPORTANT" and capitals do not make advice enforcement.
- Write broad descriptions ("helps with code") or one skill that does everything.
- Give `Bash(*)` or Edit to a reviewer; grant `bypassPermissions` to a subagent.
- Put secrets in skills, hooks or `.mcp.json` ; transcripts are plaintext. Environment variables and `userConfig: sensitive` .
- Use `prompt` or `agent` hooks for enforcement.
- Let a model-invocable skill have side effects (releases, deployments, long runs): `disable-model-invocation: true` .
- Install a plugin because a list recommends it; trial it with `--plugin-dir` first.

CONTEXT

What each mechanism costs, and when



CONSEQUENCE Twenty installed skills cost twenty descriptions on every turn even when none is used; a plugin with twelve skills you do not need is paid for on every turn.

CONTEXT

Managing memory and context efficiently

Reduce the fixed cost

`CLAUDE.md` : only what cannot be derived from the code; move procedures into skills (they cost a line until used).

Nested `CLAUDE.md` and path-scoped rules for area-specific instructions: loaded when those paths are touched.

Disable unused plugins (`/plugin disable`) and MCP servers (`/mcp`); deny skills you never want (`Skill(name)`).

`disable-model-invocation: true` removes a skill's description from the context as well.

Control the variable cost

Subagents for large reads, test output and reviews; ask for short returns.

Skill scripts print summaries (`tail -3` , PASS/FAIL), not full logs.

`/clear` between tasks; `/compact <focus>` mid-task; `/autocompact` to set the window.

Auto memory for durable project facts (`MEMORY.md` , first 200 lines load); not for anything in a file already.

Measure — `/context all` before and after every change to `.claude/` · `/skill-doctor` after a week of use · `/doctor` for unused items · `/usage` for spend by skill, agent and server

ECOSYSTEM

Where public skills and plugins come from

Official marketplace claude-plugins-official	Registered by default; browse with `/plugin` → Discover or at claude.com/plugins. Language-server plugins, vendor integrations (GitHub, GitLab, Atlassian, Linear, Slack, Sentry), workflow plugins, security guidance, output styles. Written or reviewed by Anthropic.
Community marketplace anthropics/claude-plugins-community	Over 2,000 community plugins, each pinned to a commit and passed through automated security screening. Hosting is not endorsement. `/plugin install name@claude-community`.
Anthropic skills repository anthropics/skills	Document skills (docx, pptx, xlsx, pdf) and example skills (skill-creator, mcp-builder, webapp-testing, frontend-design). Also the Agent Skills specification. `/plugin marketplace add anthropics/skills`.
Curated lists awesome-claude-code	Sections for DevOps, security, orchestration, skills, testing, linting. No formal review; a listing means someone recommended it.
Author repositories	Search GitHub for `path:SKILL.md` or `path:.claude-plugin/plugin.json`. No screening at all.
MCP servers	The vendor's own page first; the official marketplace integrations wrap many of them; `modelcontextprotocol/servers` for reference servers.

ECOSYSTEM

A shortlist for software engineering

pyright-lsp, typescript-lsp, gopls-lsp, rust-analyzer-lsp	Official. Real diagnostics and references from a language server. Needs the binary. High value, low risk.
pr-review-toolkit	Official. Specialised review agents (quality, tests, comments, errors, types) and `/review-pr`.
security-guidance	Official. PreToolUse hook that warns on insecure patterns as code is written.
commit-commands · github · sentry	Official. Commit messages from the diff; issue and PR access; error context.
skill-creator · mcp-builder · webapp-testing	anthropics/skills. Build and benchmark skills; design MCP servers; drive a browser against a local app.
superpowers (obra)	Community. Process skills: brainstorming, plans, TDD, debugging, code review, worktrees. Widely adopted; opinionated.
tdd-guard · semgrep · docker-image-scan	Community. Hooks that enforce the TDD cycle; static analysis and secret scanning; image vulnerability scans.
agnix · Claude Code Safety Net	Community. Linter for agent configuration files; a hook that catches destructive git and filesystem commands.

SKIP "100+ agents" bundles (context cost on every turn, uneven quality), duplicates of built-ins, and prompts from other tools repackaged with a SKILL.md header.

ECOSYSTEM

Vetting and trialling a public extension

Read (10 minutes)

Maintainer, last commit, releases, licence, SECURITY.md.

`plugin.json` : what it will install.

Every hook and script: they run as you. Network calls, writes outside the project, obfuscation → stop.

`.mcp.json` : which process starts, which credentials it reads.

Descriptions: do they overlap with your skills?

Evals or tests present?

Trial (15 minutes)

```
git clone <repo> /tmp/trial
git worktree add ../askdb-trial
cd ../askdb-trial
claude --plugin-dir /tmp/trial
/context all        # record the cost
# one real task; then
/skill-doctor
claude plugin eval /tmp/trial --runs 3
```

Default permission mode during the trial; narrowest tokens for any server; `git diff` before committing anything.

DECIDE Adopt at user scope · adopt at project scope · copy and modify · drop. Record the reason in one line.

SUMMARY

One line each

CLAUDE.md	Short standing facts; loaded every turn; advice, not enforcement.
Skill	A procedure with a precise description; scripts for the deterministic parts; a stop condition; tested with should and should-not cases.
Hook	A fast script at an event; `command` type; exit 2 to block; permission rules first.
Subagent	Own context, minimum tools, cheapest adequate model, fixed return format.
Plugin	The package: validated, trialled with `--plugin-dir`, installed at project scope, pinned in a marketplace.
Context	Measure with `/context all`; pay only for what is used; disable the rest.

WHAT FOLLOWS

From the terminal to the product

Built in this module

Skills that produce the configuration and evaluation files for any database; hooks that enforce the data-access rule; a reviewer subagent; evals in CI; a plugin for the team; measured context cost.

Next module

The same ideas inside the application: the Claude API and tool use; the Agent SDK with the skills and subagents you wrote today as programmatic components; an MCP server that exposes the database to any agent; evaluation of the agent against the same validation set.

CARRY FORWARD `config/databases/cloudmetrics.yaml` , `config/validation/cloudmetrics.yaml` , and the pass-rate numbers.

REFERENCES

Documentation

Skills	code.claude.com/docs/en/skills
Hooks	code.claude.com/docs/en/hooks
Subagents	code.claude.com/docs/en/sub-agents
Plugins · reference · marketplaces	code.claude.com/docs/en/plugins · /plugins-reference · /plugins/marketplace-reference · /discover-plugins
Plugin evals	code.claude.com/docs/en/plugin-evals
MCP	code.claude.com/docs/en/mcp
Memory · context window · best practices	code.claude.com/docs/en/memory · /context-window · /best-practices
Permissions · settings	code.claude.com/docs/en/permissions · /settings
Public sources	github.com/anthropics/claude-plugins-official · /anthropics/claude-plugins-community · /anthropics/skills · /hesreallyhim/awesome-claude-code · /obra/superpowers · agentskills.io

Manaranjan Pradhan · www.manaranjanp.com